

Phase 9 Structured Notes: Priority Queues

1. Current Progress Before Phase 9

Before Phase 9, you already learned many important queue ideas.

In Phase 1, you learned the basic queue rule:

FIFO = First In, First Out

This means:

The first value inserted into the queue is the first value removed from the queue.

You also learned that a normal queue has two important ends:

Queue Part	Simple Meaning
front	The side where deletion happens
rear	The side where insertion happens

So in a normal queue:

Insert at rear.
Delete from front.

In Phase 2, you learned array queues.

You compared:

Queue Type	Main Idea	Result
Single-pointer queue	Uses only rear	Deletion is slow because elements must shift
Two-pointer queue	Uses front and rear	Deletion is fast because only front moves

The most important idea was:

The two-pointer queue avoids shifting.

In Phase 3, you built a complete linear array queue in C using:

- `struct Queue`
- `size`
- `front`
- `rear`
- `Q`
- `create()`
- `enqueue()`
- `dequeue()`
- `display()`
- `malloc()`
- `free()`

In Phase 4, you implemented the same queue idea using C++ OOP.

You used:

- `class Queue`
- private data
- public methods
- constructors
- destructor
- `new[]`
- `delete[]`
- object syntax like `q.enqueue(10)`

In Phase 5, you learned the weakness of a linear array queue.

The problem was:

Memory reuse.

A linear queue wastes memory because `rear` only moves forward. Deleted spaces at the beginning of the array cannot be reused easily.

In Phase 6, you solved that problem using a circular queue.

A circular queue allows `front` and `rear` to wrap around using modulo:

```
(rear + 1) % size  
(front + 1) % size
```

In Phase 7, you learned linked-list queues.

A linked-list queue stores values inside connected nodes instead of a fixed array.

In Phase 8, you learned double ended queues, also called dequeues.

A deque is more flexible than a normal queue because insertion and deletion can happen at both ends.

Now Phase 9 introduces another special type of queue:

Priority Queue

A priority queue changes the deletion rule.

A normal queue deletes based on arrival order.

A priority queue deletes based on importance.

2. Main Goal of Phase 9

The goal of Phase 9 is to understand priority queues.

A priority queue is a queue where each element has a priority.

The element with the highest priority is removed first.

So the main question changes.

A normal queue asks:

Who came first?

A priority queue asks:

Who is most important?

This is the heart of Phase 9.

3. What Is a Priority Queue?

A priority queue is a special queue where each element has some level of importance.

In a normal queue, the first element inserted is always removed first.

Example normal queue:

Inserted order: 10, 20, 30
Removed order: 10, 20, 30

The value 10 came first, so 10 leaves first.

But in a priority queue, the most important value may leave first, even if it did not arrive first.

Example priority queue:

Element: 10, priority 3
Element: 20, priority 1
Element: 30, priority 2

If priority 1 is the highest priority, then 20 should be removed first.

Why?

Because 20 has priority 1, which is more important than priority 2 or priority 3.

4. Real-Life Example of a Priority Queue

A good real-life example is a hospital emergency room.

Imagine three patients arrive:

Patient	Arrival Order	Problem	Priority
Patient A	1st	Small injury	3
Patient B	2nd	Serious emergency	1
Patient C	3rd	Medium problem	2

In a normal queue, Patient A would be treated first because Patient A arrived first.

But in a priority queue, Patient B should be treated first because Patient B has the most serious problem.

So the treatment order is:

Patient B, Patient C, Patient A

This shows the main idea:

Priority can be more important than arrival order.

5. Priority Rule Used in These Notes

In these notes, we use this rule:

Smaller number = higher priority

That means:

Priority 1 is higher than priority 2.
Priority 2 is higher than priority 3.
Priority 3 is higher than priority 4.

So priority **1** is the most important.

Example:

Priority Number	Meaning
1	Highest priority
2	Medium priority
3	Lower priority

This rule is very important.

When comparing priorities in Phase 9, remember:

The smaller priority number wins.

6. Priority Queue vs Normal Queue vs Deque

A priority queue is different from a normal queue and a deque.

Structure	Deletion Rule
Normal queue	Delete the oldest element first
Deque	Can delete from front or rear
Priority queue	Delete the highest-priority element first

A normal queue follows strict FIFO.

A deque is flexible because it can work from both ends.

A priority queue focuses on importance.

7. Important Difference Between FIFO and Priority

In a normal queue:

The first inserted value leaves first.

In a priority queue:

The highest-priority value leaves first.

Example:

Insert A with priority 3
Insert B with priority 1
Insert C with priority 2

Even though **A** arrived first, **B** is deleted first because it has the highest priority.

Deletion order:

B, C, A

Why?

Because:

Priority 1 is higher than priority 2.
Priority 2 is higher than priority 3.

8. Does a Priority Queue Always Ignore FIFO?

Not completely.

A priority queue mainly follows priority.

But if two elements have the same priority, then FIFO can still apply between them.

Example:

Insert A with priority 1
Insert F with priority 1

Both **A** and **F** have the same priority.

Since **A** arrived before **F**, **A** should leave before **F**.

So inside the same priority level:

FIFO is preserved.

Important rule:

Different priority: highest priority leaves first.
Same priority: first arrived leaves first.

Part A: Limited Priority Queue Using Multiple Queues

9. What Is a Limited Priority Queue?

A limited priority queue is used when the number of priority levels is small and fixed.

For example, suppose we only have three priority levels:

```
Priority 1 = highest  
Priority 2 = medium  
Priority 3 = lowest
```

Because there are only three priority levels, we can create three normal queues.

```
Queue 1 stores priority 1 elements.  
Queue 2 stores priority 2 elements.  
Queue 3 stores priority 3 elements.
```

Each priority level gets its own queue.

10. Structure of Multiple-Queue Priority Queue

The structure looks like this:

```
Priority 1 Queue: [ elements with priority 1 ]  
Priority 2 Queue: [ elements with priority 2 ]  
Priority 3 Queue: [ elements with priority 3 ]
```

If a new element has priority **1**, insert it into Queue 1.

If a new element has priority **2**, insert it into Queue 2.

If a new element has priority **3**, insert it into Queue 3.

11. Insertion in Limited Priority Queue

Insertion is simple.

To insert an element:

1. Look at its priority.
2. Go to the queue for that priority.
3. Insert the element at the rear of that queue.

Example:

Insert A with priority 1

A goes into Queue 1.

Queue 1: A
Queue 2: empty
Queue 3: empty

Now insert:

Insert D with priority 3

D goes into Queue 3.

Queue 1: A
Queue 2: empty
Queue 3: D

Now insert:

Insert F with priority 1

F goes into Queue 1 after A.

Queue 1: A, F
Queue 2: empty
Queue 3: D

12. Deletion in Limited Priority Queue

Deletion checks the highest-priority queue first.

Since smaller priority number means higher priority, we check Queue 1 first.

Deletion logic:

```
If Queue 1 is not empty, delete from Queue 1.  
Else if Queue 2 is not empty, delete from Queue 2.  
Else if Queue 3 is not empty, delete from Queue 3.  
Else the priority queue is empty.
```

So we always try to delete from the most important non-empty queue.

13. Dry Run: Limited Priority Queue

Insert these elements:

```
A with priority 1  
D with priority 3  
F with priority 1
```

After insertion:

```
Queue 1: A, F  
Queue 2: empty  
Queue 3: D
```

Now delete one element.

Step 1:

```
Check Queue 1.
```

Queue 1 is not empty.

So delete from Queue 1.

Deleted value:

A

Now the queues are:

Queue 1: F
Queue 2: empty
Queue 3: D

Now delete again.

Check Queue 1.

Queue 1 is still not empty.

Deleted value:

F

Now the queues are:

Queue 1: empty
Queue 2: empty
Queue 3: D

Now delete again.

Queue 1 is empty.

Queue 2 is empty.

Queue 3 is not empty.

Deleted value:

D

Final deletion order:

A, F, D

14. Why Did F Leave Before D?

D arrived before F.

But D has priority 3.

F has priority 1.

Since priority 1 is higher than priority 3, F leaves before D.

So the priority queue does not simply follow arrival order.

It follows importance first.

15. FIFO Inside the Same Priority

In the same dry run:

A with priority 1
F with priority 1

Both A and F have the same priority.

A arrived first.

So A leaves before F.

This proves that FIFO still works inside the same priority level.

16. Time Complexity of Limited Priority Queue

Insertion is usually fast.

If we already know the priority, we directly insert into the correct queue.

So insertion is usually:

$O(1)$

Deletion depends on how many priority levels we may need to check.

If there are 3 priority levels, we may check up to 3 queues.

If there are k priority levels, deletion is:

$$O(k)$$

Where k means the number of priority levels.

If k is small, this method is simple and useful.

Part B: Element Itself as Priority

17. What Does “Element Itself as Priority” Mean?

Sometimes we do not store a separate priority number.

Instead, the value itself decides the priority.

In these notes, we use this rule:

Smaller value = higher priority

Example:

6, 8, 3

The smallest value is 3.

So 3 has the highest priority.

Therefore, 3 should be deleted first.

18. Example: Element as Priority

Suppose the priority queue contains:

6, 8, 3

Using the rule:

Smaller value = higher priority

Compare values:

Value	Priority Meaning
3	Highest priority
6	Next priority
8	Lowest priority among these

So the deletion order should be:

3, 6, 8

This is different from a normal queue.

If this were a normal queue and values were inserted as 6, 8, 3, the deletion order would be:

6, 8, 3

But because this is a priority queue, the smallest value leaves first.

19. Two Array-Based Solutions

For element-as-priority, we can implement the priority queue using an array.

The notes discuss two simple solutions:

Solution	Insert	Delete	Main Idea
Solution A	Fast	Slow	Insert at the end, search during deletion
Solution B	Slow	Fast	Keep array sorted, delete from the end

These two solutions show an important programming tradeoff.

A tradeoff means:

If one operation becomes faster, another operation may become slower.

Solution A makes insertion easy, but deletion takes more work.

Solution B makes deletion easy, but insertion takes more work.

Part C: Solution A — Insert Fast, Delete Slow

20. Main Idea of Solution A

In Solution A, insertion is very simple.

We do not sort the array.

We just insert the new value at the end.

Example:

```
Insert 6 → [6]
Insert 8 → [6, 8]
Insert 3 → [6, 8, 3]
```

The array is not sorted.

It keeps values in the order they were inserted.

21. Why Is Insertion Fast in Solution A?

Insertion only does two things:

1. Put the new value at the next empty position.
2. Increase the size.

No searching is needed.

No sorting is needed.

No shifting is needed during insertion.

So insertion is:

$O(1)$

$O(1)$ means constant time.

The operation does not become slower just because the queue has more elements.

22. Why Is Deletion Slow in Solution A?

Deletion is slow because the array is not sorted.

To delete the highest-priority element, we must search the whole array to find the smallest value.

Example:

Array: [6, 8, 3]

We cannot immediately know where the highest-priority value is.

So we must scan the array.

Search process:

```
6 → current smallest  
8 → bigger than 6, ignore  
3 → smaller than 6, new smallest
```

The smallest value is 3.

So delete 3.

Because we may need to check every element, deletion is:

$O(n)$

23. Solution A Dry Run

Start with an empty array:

```
[ ]
```

Insert **6**:

```
[6]
```

Insert **8**:

```
[6, 8]
```

Insert **3**:

```
[6, 8, 3]
```

Now delete.

We search for the smallest value.

Step-by-step:

```
Start: min = 6  
Compare 8 with 6 → 8 is bigger, so min stays 6  
Compare 3 with 6 → 3 is smaller, so min becomes 3
```

Highest-priority value:

```
3
```

Delete **3**.

Final array:

```
[6, 8]
```

Deleted value:

3

24. Important Step After Deleting in Solution A

After deleting the smallest value, we should remove its space from the array.

If the deleted value is in the middle, we must shift the remaining elements left.

Example:

Array: [6, 3, 8]

The smallest value is 3, which is at index 1.

Delete 3.

Now 8 must shift left.

Final array:

[6, 8]

If we do not shift, the array may have a gap.

That would make the queue incorrect.

25. Solution A Code

```
#include <iostream>
using namespace std;

void insertFast(int arr[], int& size, int x) {
    arr[size] = x;
    size++;
}

int deleteSlow(int arr[], int& size) {
```

```

if (size == 0) {
    cout << "Priority queue is empty\n";
    return -1;
}

int minIndex = 0;

// Find the smallest value
for (int i = 1; i < size; i++) {
    if (arr[i] < arr[minIndex]) {
        minIndex = i;
    }
}

int deleted = arr[minIndex];

// Shift elements left after deletion
for (int i = minIndex; i < size - 1; i++) {
    arr[i] = arr[i + 1];
}

size--;
return deleted;
}

```

26. Explanation of Solution A Code

insertFast()

```

void insertFast(int arr[], int& size, int x) {
    arr[size] = x;
    size++;
}

```

This function inserts quickly.

Line by line:

```
arr[size] = x;
```

This stores the new value at the next empty position.

```
size++;
```

This increases the number of elements.

```
deleteSlow()
```

```
int minIndex = 0;
```

This assumes the first value is currently the smallest.

```
for (int i = 1; i < size; i++) {  
    if (arr[i] < arr[minIndex]) {  
        minIndex = i;  
    }  
}
```

This loop searches for the smallest value.

```
int deleted = arr[minIndex];
```

This saves the value that will be deleted.

```
for (int i = minIndex; i < size - 1; i++) {  
    arr[i] = arr[i + 1];  
}
```

This shifts elements left so there is no gap.

```
size--;
```

This reduces the number of elements.

```
return deleted;
```

This returns the deleted value.

27. Complexity of Solution A

Operation	Time Complexity	Reason
Insert	$O(1)$	Put value at the end
Delete	$O(n)$	Must search for the smallest value

Solution A is useful when insertion should be fast.

But deletion is slower.

Part D: Solution B — Insert Slow, Delete Fast

28. Main Idea of Solution B

In Solution B, we keep the array sorted.

The array is kept in descending order.

Descending order means:

Largest value first, smallest value last.

Example:

[8, 6, 3]

Why descending?

Because in these notes:

Smaller value = higher priority

So the smallest value should be deleted first.

If the smallest value is always at the end, deletion becomes easy.

29. Why Is Deletion Fast in Solution B?

If the array is sorted in descending order, the smallest value is at the end.

Example:

```
[8, 6, 3]
```

The highest-priority value is **3**.

It is already at the end.

So deletion simply removes the last element.

This is fast:

```
O(1)
```

30. Why Is Insertion Slow in Solution B?

Insertion is slow because the array must stay sorted.

When a new value is inserted, we must find the correct position for it.

Sometimes we must shift elements to make space.

Example:

Current array:

```
[8, 6, 3]
```

Insert **7**.

To keep descending order, the result should be:

```
[8, 7, 6, 3]
```

So **6** and **3** may need to move.

Because insertion may shift many elements, insertion is:

$O(n)$

31. Solution B Dry Run

Start with an empty array:

[]

Insert 6:

[6]

Insert 8.

We want descending order.

Since 8 is larger than 6, it goes before 6.

[8, 6]

Insert 3.

Since 3 is smaller than both 8 and 6, it goes at the end.

[8, 6, 3]

Now delete.

The smallest value is at the end.

Deleted value:

3

Final array:

[8, 6]

32. Solution B Code

```
#include <iostream>
using namespace std;

void insertSlow(int arr[], int& size, int x) {
    int i = size - 1;

    // Keep array in descending order
    while (i >= 0 && arr[i] < x) {
        arr[i + 1] = arr[i];
        i--;
    }

    arr[i + 1] = x;
    size++;
}

int deleteFast(int arr[], int& size) {
    if (size == 0) {
        cout << "Priority queue is empty\n";
        return -1;
    }

    size--;
    return arr[size];
}
```

33. Explanation of Solution B Code

insertSlow()

```
int i = size - 1;
```

This starts from the last valid element.

```
while (i >= 0 && arr[i] < x) {  
    arr[i + 1] = arr[i];  
    i--;  
}
```

This loop shifts smaller values to the right.

Why?

Because we want larger values to stay toward the front.

```
arr[i + 1] = x;
```

This places the new value in the correct sorted position.

```
size++;
```

This increases the number of elements.

`deleteFast()`

```
if (size == 0) {  
    cout << "Priority queue is empty\n";  
    return -1;  
}
```

This checks whether the priority queue is empty.

```
size--;  
return arr[size];
```

This removes and returns the last element.

Because the smallest value is at the end, this deletes the highest-priority element.

34. Complexity of Solution B

Operation	Time Complexity	Reason
Insert	$O(n)$	Must find correct position and may shift elements
Delete	$O(1)$	Remove the last element

Solution B is useful when deletion should be fast.

But insertion is slower.

Part E: Comparing Solution A and Solution B

35. Main Difference

Solution A keeps the array unsorted.

Solution B keeps the array sorted.

Solution	Array Style	Insert	Delete
Solution A	Unsorted	Fast	Slow
Solution B	Sorted descending	Slow	Fast

36. Why Solution A Has Fast Insert

Solution A does not care about order during insertion.

It simply adds the new value at the end.

Example:

```
Insert 6 → [6]
Insert 8 → [6, 8]
Insert 3 → [6, 8, 3]
```

This makes insertion fast.

But later, deletion must search for the smallest value.

37. Why Solution B Has Fast Delete

Solution B does extra work during insertion.

It keeps the array sorted in descending order.

Example:

[8, 6, 3]

The smallest value is already at the end.

So deletion simply removes the last value.

This makes deletion fast.

38. Complexity Comparison Table

Method	Insert	Delete	Main Idea
Limited priority queues	$O(1)$ usually	$O(\text{number of priority levels})$	Use separate queues for priority levels
Solution A: unsorted array	$O(1)$	$O(n)$	Insert at end, search during delete
Solution B: sorted array	$O(n)$	$O(1)$	Sort during insert, delete from end

39. Which Solution Is Better?

There is no single best solution for all situations.

It depends on what you need more.

If your program performs many insertions and few deletions, Solution A may be better.

Why?

Because insertion is fast.

Solution A insert = $O(1)$

If your program performs many deletions and you want deletion to be fast, Solution B may be better.

Why?

Because deletion is fast.

Solution B delete = $O(1)$

So the choice depends on the situation.

40. Future Improvement: Heap

These notes mention simple priority queue implementations.

But there is a more efficient structure commonly used for priority queues:

Heap

A heap can make priority queue operations more efficient than these simple array methods.

You do not need to deeply understand heaps in Phase 9.

Just remember:

Heap is a future improvement for priority queues.

Part F: Complete Beginner-Friendly Program

41. Complete Program Showing Solution A and Solution B

```
#include <iostream>
using namespace std;

// Solution A: Insert fast, delete slow
```

```

void insertFast(int arr[], int& size, int x) {
    arr[size] = x;
    size++;
}

int deleteSlow(int arr[], int& size) {
    if (size == 0) {
        cout << "Priority queue is empty\n";
        return -1;
    }

    int minIndex = 0;

    // Find smallest value
    for (int i = 1; i < size; i++) {
        if (arr[i] < arr[minIndex]) {
            minIndex = i;
        }
    }

    int deleted = arr[minIndex];

    // Shift remaining elements left
    for (int i = minIndex; i < size - 1; i++) {
        arr[i] = arr[i + 1];
    }

    size--;
    return deleted;
}

// Solution B: Insert slow, delete fast

void insertSlow(int arr[], int& size, int x) {
    int i = size - 1;

    // Keep array in descending order
    while (i >= 0 && arr[i] < x) {
        arr[i + 1] = arr[i];
        i--;
    }

    arr[i + 1] = x;
    size++;
}

int deleteFast(int arr[], int& size) {
    if (size == 0) {

```

```

        cout << "Priority queue is empty\n";
        return -1;
    }

    size--;
    return arr[size]; // smallest value is at the end
}

int main() {
    int a[10];
    int sizeA = 0;

    insertFast(a, sizeA, 6);
    insertFast(a, sizeA, 8);
    insertFast(a, sizeA, 3);

    cout << "Solution A deleted: " << deleteSlow(a, sizeA) << endl;

    int b[10];
    int sizeB = 0;

    insertSlow(b, sizeB, 6);
    insertSlow(b, sizeB, 8);
    insertSlow(b, sizeB, 3);

    cout << "Solution B deleted: " << deleteFast(b, sizeB) << endl;

    return 0;
}

```

42. Expected Output

```

Solution A deleted: 3
Solution B deleted: 3

```

Both solutions delete **3** first.

Why?

Because in these notes:

Smaller value = higher priority

So 3 has higher priority than 6 and 8.

43. Full Program Dry Run

Solution A Dry Run

Start:

```
Array A = empty  
sizeA = 0
```

Insert 6:

```
Array A = [6]  
sizeA = 1
```

Insert 8:

```
Array A = [6, 8]  
sizeA = 2
```

Insert 3:

```
Array A = [6, 8, 3]  
sizeA = 3
```

Now call:

```
deleteSlow(a, sizeA)
```

Search for smallest:

```
6 → current smallest  
8 → not smaller  
3 → smaller than 6
```

Delete 3.

Output:

```
Solution A deleted: 3
```

Solution B Dry Run

Start:

```
Array B = empty  
sizeB = 0
```

Insert **6**:

```
Array B = [6]  
sizeB = 1
```

Insert **8**:

```
Array B = [8, 6]  
sizeB = 2
```

Insert **3**:

```
Array B = [8, 6, 3]  
sizeB = 3
```

Now call:

```
deleteFast(b, sizeB)
```

The smallest value is already at the end.

Delete **3**.

Output:

```
Solution B deleted: 3
```

Part G: Common Beginner Mistakes

44. Mistake 1: Thinking a Priority Queue Is Always FIFO

A priority queue is not always FIFO.

A normal queue deletes the oldest element first.

A priority queue deletes the highest-priority element first.

Wrong idea:

The first inserted element must always leave first.

Correct idea:

The highest-priority element leaves first.

FIFO only applies when two elements have the same priority.

45. Mistake 2: Reversing the Priority Rule

In these notes:

Smaller number = higher priority

So priority **1** is higher than priority **3**.

Wrong idea:

Priority 3 is higher than priority 1.

Correct idea:

Priority 1 is higher than priority 3.

For element-as-priority examples:

3 has higher priority than 8.

Why?

Because 3 is smaller than 8.

46. Mistake 3: Forgetting FIFO Within the Same Priority

If two elements have the same priority, the one that arrived first should leave first.

Example:

A with priority 1
F with priority 1

Deletion order should be:

A, F

Because both have the same priority, arrival order matters.

47. Mistake 4: Forgetting to Shift in Solution A

In Solution A, after deleting the smallest value, the array may need shifting.

Example:

Array: [6, 3, 8]

Delete 3.

If we do not shift, the array may have an empty gap.

Correct final array:

[6, 8]

48. Mistake 5: Sorting in the Wrong Order in Solution B

In Solution B, the array must be sorted in descending order.

Correct:

```
[8, 6, 3]
```

Why?

Because the smallest value should be at the end.

Then deletion can remove the end value quickly.

Wrong order:

```
[3, 6, 8]
```

If the smallest value is at the front, then deleting from the end would remove the wrong value.

49. Mistake 6: Forgetting to Check Empty Before Deletion

Before deleting, always check whether the priority queue is empty.

If the queue is empty, there is nothing to delete.

That is why the code checks:

```
if (size == 0) {  
    cout << "Priority queue is empty\n";  
    return -1;  
}
```

This prevents invalid deletion.

Part H: Final Summary

50. Main Ideas of Phase 9

A priority queue is a special queue where deletion is based on priority.

A normal queue asks:

Who came first?

A priority queue asks:

Who is most important?

In these notes:

Smaller number = higher priority

Priority queues can be implemented in different ways.

The three main methods discussed are:

1. Multiple queues for fixed priority levels
2. Unsorted array: fast insert, slow delete
3. Sorted array: slow insert, fast delete

51. Final Comparison

Concept	Simple Meaning
Priority queue	Deletes the highest-priority element first
Smaller priority number	Means higher priority in these notes
Multiple queues	One queue for each priority level
FIFO inside same priority	Same-priority elements leave in arrival order
Solution A	Insert fast, delete slow
Solution B	Insert slow, delete fast

Concept

Simple Meaning

Heap

Future better implementation for priority queues

52. One-Sentence Final Takeaway

A priority queue is a queue where the most important element leaves first, not necessarily the element that arrived first.

53. Quick Self-Test

Question 1

If smaller number means higher priority, which priority is highest?

Priority 1, priority 2, or priority 3?

Answer:

Priority 1

Question 2

If the elements are:

6, 8, 3

Which value is deleted first?

Answer:

3

Why?

Because smaller value means higher priority.

Question 3

In Solution A, why is deletion slow?

Answer:

Because the array is unsorted, so we must search for the smallest value.

Question 4

In Solution B, why is insertion slow?

Answer:

Because the array must stay sorted, so elements may need to shift.

Question 5

If two elements have the same priority, which one leaves first?

Answer:

The one that arrived first.

54. End of Phase 9

You now understand priority queues.

You learned:

- what a priority queue is
- how it differs from a normal queue
- why priority can beat arrival order
- why same-priority elements still follow FIFO
- how multiple queues can represent fixed priorities
- how an unsorted array gives fast insertion and slow deletion
- how a sorted array gives slow insertion and fast deletion
- the time complexity of each method
- common beginner mistakes

You are now ready for Phase 10, where queues are implemented using two stacks and all queue ideas are integrated together.